UNIVERSITY OF CALIFORNIA
SANTA CRUZ

AN END-TO-END OPEN-SOURCE FLOW FOR FLIP-FLOP TO TWO-PHASE LATCH CONVERSION WITH SYNTHESIS OPTIMIZATIONS AND CLOCK GATING

A project submitted in partial satisfaction of the
requirements for the degree of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE AND ENGINEERING

by

Paolo Pedroso

June 2026

The Bachelor's Project of Paolo Pedroso
is approved:

Professor Matthew Guthaus

Professor Dustin Richmond

Professor Jose Renau

Table of Contents

List of Figures

List of Tables

Abstract

An End-to-End Open-Source Flow for Flip-Flop to Two-Phase Latch Conversion with Synthesis Optimizations and Clock Gating

by

Paolo Pedroso

Two-phase clocking offers timing-margin and clock-flexibility advantages, but its adoption is limited by the absence of automation in modern design flows, leaving it rare in practice. This thesis completes the first open-source fully automated two-phase clocking flow in OpenROAD Flow Scripts (ORFS), extending a prior front-end methodology into a complete RTL-to-GDS flow. The contributions span the flow end-to-end: an optimized ABC retime script interleaving AIG restructuring then retiming, a clock-gated variant alongside the existing recirculation-mux variant, dual clock tree synthesis for the non-overlapping Φ_1/Φ_2 clocks via TritonCTS, and a two-coloring static verification pass in OpenROAD's Python API. Across GCD, AES, RISCV32I, and JPEG clock-gated variants, the optimized retime scripts reduce total power by an average of 12.3% and design area by 12.9% over the original retime script while closing hold timing, peaking at 27.8% power and 20.2% area on RISCV32I. The clock-gated variant averages 29.2% lower power and 50% fewer latches than recirculation muxes. Against flip-flop baselines, the RISCV32I flip-flop baseline fails timing at 6.4 ns while both two-phase variants close timing at the same period through time borrowing; all designs retain unused time-borrow capacity, indicating further frequency headroom.

Acknowledgments

I would like to thank Lee Way, Matthew Guthaus, members of VLSI-DA, and my family.

Chapter 1

Introduction

The two-phase clocking scheme dates back to the early 1970s, with early microprocessors such as the Intel 8008 among its first prominent implementations. As technology nodes shrank and operating frequencies increased through the 1980s, two-phase systems introduced complexities in timing verification, particularly concerning hold time violations, race conditions, and precise control of non-overlapping clock phases [3]. Managing clock skew with multiple phases then became increasingly difficult as circuit complexity rose [10]. By the time logic synthesis, automated physical design, and modern STA became standard practice, two-phase clocking was no longer a focus, and the entire EDA ecosystem shifted to single-phase, edge-triggered flip-flop designs. Two-phase clocking is rarely found in modern integrated circuits, and the industry's widespread adoption of flip-flop-based designs combined with EDA tools optimized for this paradigm has made two-phase systems largely obsolete. The dominance of single-phase, edge-triggered flip-flops has left two-phase clocked latches largely overlooked in

modern digital design, a gap that warrants renewed investigation. Two-phase clocking offers additional benefits including flexibility, reduced clock skew sensitivity, and time borrowing capabilities that enable higher operating frequencies [11]. The paradigm allows tunability, especially for fixing hold times, which can cause permanent failure. For example, many of the open-source designs on the Google-Skywater shuttle run failed because of hold time problems in the GPIO scan chain. In two-phase designs, widening the Φ_1 and Φ_2 gap relaxes the hold constraint without the cost of reducing clock frequency.

Latches themselves offer inherent power and area advantages over flip-flops [3], as they use simpler level-sensitive circuits with fewer transistors. The widespread dominance of flip-flop-based designs, combined with the availability of open-source EDA tools such as OpenROAD, Yosys, and ABC [22, 9, 5, 8], presents a unique opportunity to democratize access to unconventional design methodologies. By leveraging these tools, this thesis introduces two-phase clocking into the OpenROAD ecosystem, enabling the designer to transform flip-flop-based RTL netlists into two-phase non-overlapping latch-based implementations without manual expert intervention, making an otherwise inaccessible design paradigm openly available for research, education, and exploration. This thesis builds on the front-end conversion methodology developed in prior work by Wang and Guthaus [23], which established flip-flop duplication, transformation, ABC retiming, and equivalence checking for the recirculation-mux variant. The contributions of this thesis are:

- **Two-Phase Clock-Gated Variant:** A clock-gated flip-flop-to-latch conversion is introduced alongside the existing recirculation-mux variant, reducing latch count and power through clock gating of the Φ_2 latch rather than recirculation logic.

- **Optimized ABC Synthesis Script:** An iterative retiming and logic-restructuring script is developed to replace the original retiming script built by Wang, targeting logic cell reduction and timing closure for two-phase schemes.

- **Dual Clock Tree Synthesis:** Dual clock tree synthesis is performed using TritonCTS [19] for the 180° out-of-phase, non-overlapping Φ_1 and Φ_2 clocks, with options tuned to account for asymmetric latch insertion delays introduced by clock gating.

- **Two-Coloring Static Verification:** A two-coloring static verification pass [24] is developed using OpenROAD's Python API to validate two-phase latch implementations at the finishing step.

- **End-to-End RTL-to-GDS Evaluation:** The complete flow is evaluated on GCD, AES, RISCV32I, and JPEG against flip-flop baselines, demonstrating timing closure through time borrowing. The optimized ABC retiming script is also compared against the original retime script across the same designs.

Together, these contributions provide the first end-to-end, open-source, automated path from a flip-flop-based RTL design to a fully implemented two-phase clocked system, making two-phase clocking accessible and reproducible for the broader research

community. The remainder of this thesis is organized as follows. Chapter 2 reviews the timing constraints of two-phase clocking, time borrowing, retiming, and the Yosys ABC synthesis flow. Chapter 3 presents the proposed methodology, with Section 3.1 describing the front-end synthesis and clock conversion and Section 3.2 describing the back-end placement, clock tree synthesis, and verification. Chapter 4 evaluates the flow across three experiments. Section 4.1 performs a logic synthesis comparison of the ABC retime scripts on RISCV32I, Section 4.2 extends this to a post-PnR comparison across all four designs, and Section 4.3 provides the end-to-end comparison between the flip-flop baselines and two-phase variants.

Chapter 2

Background

2.1 Timing Constraints

In flip-flop designs, setup violations can be resolved by reducing the clock frequency, but hold violations cannot since hold time is independent of the clock period. Two-phase clocking offers more flexibility, because widening the spacing between the two clock pulses improves hold slack at the cost of some setup slack. This tunability makes two-phase clocking more resilient to clock skew and hold time issues than flip-flop-based designs. Another advantage of level-sensitive latches over flip-flops is time borrowing, in which a stage that cannot meet timing borrows slack from the next stage. Harris et al. [6] formalize the timing constraints of latch-based systems, presenting the setup requirement:

$$A_i^c + \Delta_{DCi} + t_{skew}^{pi,c} \leq T_{P_i} \tag{2.1}$$

where A_i^c is the arrival time of a signal launched by clock c at the input to latch i, Δ_{DCi} is the setup time requirement for latch i between the data pin and the falling clock edge, $t_{skew}^{pi,c}$ is the skew between clock c and the clock phase p_i of latch i, and T_{P_i} is the time at which the sampling clock edge arrives. As Eq. (2.1) shows, slowing the clock causes the sampling edge to arrive later, relaxing the setup constraint. Time borrowing from the next latch has the same effect, as it also delays the sampling clock edge.

As for the hold requirement:

$$\delta_{DQi} + \delta_{ji} + S_{p_j p_i} \geq T_i + \Delta_{CDi} + t_{skew}^{pi,pj} - T_c \tag{2.2}$$

where δ_{DQi} is the minimum input-to-output propagation delay through latch i, δ_{ji} is the minimum combinational delay between latches j and i, $S_{p_j p_i}$ is the phase shift between p_j and p_i, T_i is the clock pulse width, Δ_{CDi} is the hold time requirement, $t_{skew}^{pi,pj}$ is the skew between phases p_i and p_j, and T_c is the clock period [6]. As Eq. (2.2) shows, decreasing T_i (i.e., widening the spacing between clock phases) relaxes the hold constraint, while time borrowing relaxes the setup constraint [6].

2.2 Time Borrowing

Time-borrowing is a technique enabled by two-phase clocking that can only be applied to latches. Because latches are level-sensitive, if a combinational path cannot meet timing to the next stage, the current stage can *borrow* time from the next stage (assuming non-overlapping clocks). As shown in Fig. 2.1, with a 10ns period for both Φ_1 and Φ_2, if the first stage has a combinational path of 7ns, it can borrow 2ns from

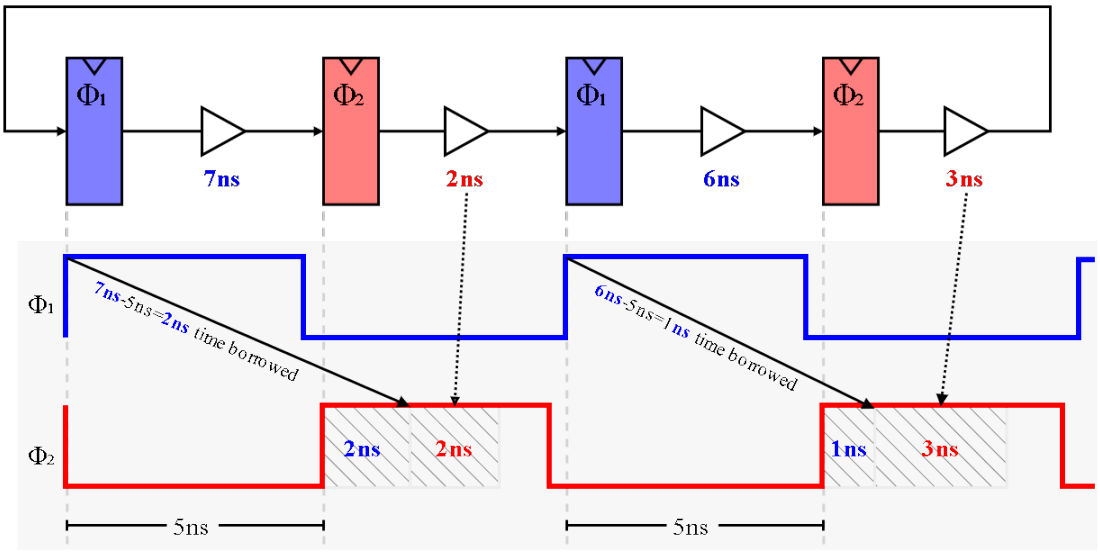


Figure 2.1: Time-borrowing example.

Time-borrowing example with Φ_1 and Φ_2 periods of 10ns.

the next stage; since the next stage's combinational path is only 2ns, timing is still met without race conditions.

2.3 Retiming

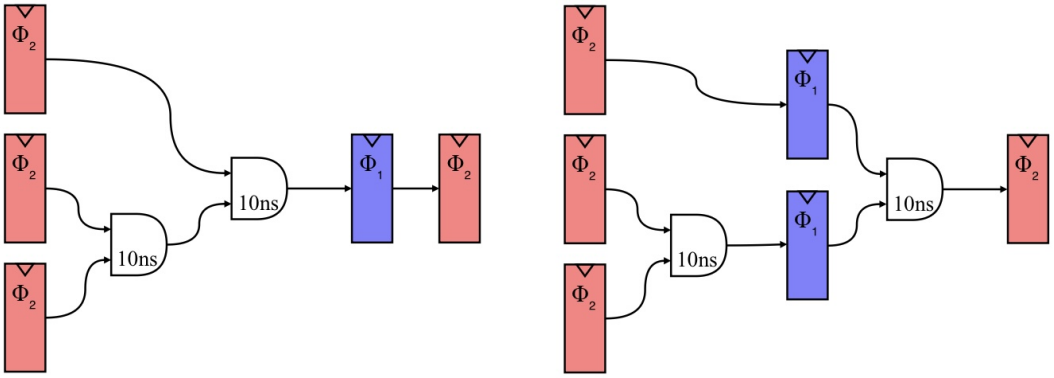
Since the framework involves a series of technology mapping and retiming steps that progressively transform and rename cells throughout the flow, this thesis introduces the following naming conventions similar to Yosys [25] to track cell types across each stage:

- $DFF\_base_{\Phi_n}$: Base posedge D flip-flop(s) (i.e. *_DFF_P_*) prior to mapping to latches, where n denotes the clock phase (Φ_1 or Φ_2).

- *DFF_fvar*$_{\Phi_n}$: Full-variant posedge D flip-flop(s) (e.g. *_DFFE_PP_*) prior to mapping to *DFF_base*$_{\Phi_n}$, where n denotes the clock phase (Φ_1 or Φ_2).

Retiming moves registers across combinational logic nodes while preserving output functionality. The algorithm guarantees that every path through the combinational network passes through the register boundary exactly once [3].

Rather than retiming directly on latch-based designs, it is more practical to first duplicate flip-flops, retime the duplicated flip-flops, and then later transform them into latch-based implementations, inspired by Singh et al. [7]. This is safe because a *_DFF_P_* is functionally equivalent to a two-phase latch pair, if both latches are positive-enable. The *DFF_base*$_{\Phi_1}$ captures the state, and the *DFF_base*$_{\Phi_2}$ outputs the state. Since the two clocks are non-overlapping and 180° out of phase, the *DFF_base*$_{\Phi_2}$ is dependent on the output state of the *DFF_base*$_{\Phi_1}$, allowing the *DFF_base*$_{\Phi_1}$ to be freely retimed across combinational logic behind *DFF_base*$_{\Phi_2}$ without violating the phase relationship.



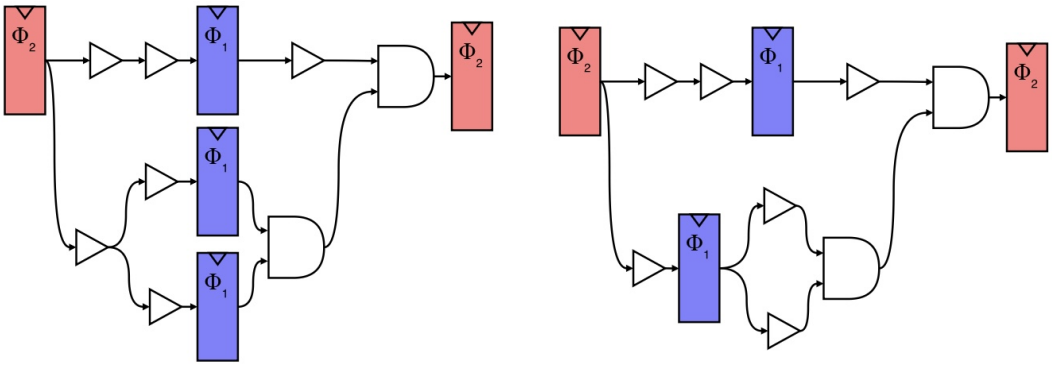
(a) Before min-delay retiming.

(b) After min-delay retiming.

Figure 2.2: Minimum-delay retiming.

2.3.1 Minimum-delay retiming

Minimum-delay retiming redistributes registers forward across combinational logic to balance path depths across phases, reducing the critical path and contributing to frequency improvement. As shown in Fig. 2.2a, the critical path is 20ns prior to retiming, yielding a maximum frequency of 50MHz. After minimum-delay retiming, Fig. 2.2b demonstrates that the critical path is halved, achieving a maximum frequency of 100MHz.



(a) Before min-area retiming. (b) After min-area retiming.

Figure 2.3: Minimum-area retiming.

2.3.2 Minimum-area retiming

Minimum-area retiming consolidates the register boundary to minimize total flip-flop count while maintaining the legal Φ_1/Φ_2 phase separation at the new register positions. As observed in Fig. 2.3a, registers are distributed across multiple stages of the combinational logic path. After minimum-area retiming, Fig. 2.3b demonstrates that

the register count is reduced by merging boundaries, while the Φ_1/Φ_2 phase separation is preserved at the consolidated register positions.

2.4 ABC Synthesis

ABC is an open-source logic optimization and technology mapping tool from UC Berkeley, integrated into the Yosys Open Synthesis Suite as the default technology mapper [8]. All algorithms made by ABC operate on the And-Inverter Graph (AIG), a directed acyclic graph of two-input AND gates with complemented edges for inversion [12]. The uniform two-input AND structure with structural sharing across subcircuits enables efficient local rewriting and technology mapping [1]. ABC's circuit optimization commands are best chained in succession to achieve targeted results. For example, the alias command `compress2`, which repeatedly applies the commands `rewrite`, `refactor`, and `balance` has been proven to reduce node count, since each pass exposes new optimization opportunities for the next [12]. In ORFS, there are two default ABC scripts, `abc_speed` focuses on timing optimizations and `abc_area` focuses on area optimizations [21, 20]. ORFS defaults to `abc_speed` if ORFS environment variable `ABC_AREA` is not set to 1. The default ORFS `abc_speed` script focuses primarily on delay optimization through rewriting and technology mapping passes, without iterative retiming. The original ABC script used for the front-end of the two-phase project employs only a single `dretime` and `retime -M 5` sequence for retiming, which restricts ABC to retiming only without applying any AIG restructuring passes that could exploit the new optimization opportu-

nities created by retiming. This thesis extends the original retime script by interleaving `compress2`-style restructuring passes then retiming, optimizing both delay and area before each retime invocation. The remainder of this section analyzes the commands used in the original ABC retime script and its optimized counterpart; full command and flag listings for `abc_speed`, `ret_base`, and the optimized scripts are provided in Appendix A.

2.4.1 ABC Speed Script

The `abc_speed` used in ORFS runs a delay-driven, choice-aware logic optimization and standard cell mapping flow on ABC's AIG package, the GIA, which the `&` prefix on commands refers to. The full commands are listed in Table A.3, Appendix A. The script begins with `&dch` to build an AIG with structural choices, then `&nf` produces a first mapping against the loaded liberty library. The script then iterates the sequence `&syn2; &if -g -K 6; &synch2; &nf; &st` five times, alternating delay-oriented AIG synthesis, SOP-balancing, choice regeneration, and remapping so that each remap exposes new opportunities for the next AIG pass [13]. After the final mapping, buffering, sizing, and timing commands finalize the netlist (`buffer -c; topo; stime -c; upsize -c; dnsize -c`).

2.4.2 ABC Retime Script

The original retime script, `ret_base`, normalizes the registers in the structural-hashed network with initial states to zero (`zero`), runs a min-area retiming pass

(`dretime`), then performs a combined min-area and min-delay retiming (`retime -M 5 -o`), and finally remaps to the standard cell library (`map`) [4, 2]. Mode 5 selected by `-M 5` chains mode 3 (forward and backward min-area retiming) followed by mode 4 (forward and backward min-delay retiming). The full set of commands is listed in Table A.1, Appendix A.

2.4.3 Optimized ABC Retime Script

The original retime script applies retiming only, with no further circuit optimization. This thesis extends it with an area and delay-oriented strategy that interleaves retiming with combinational AIG optimization presented in Table A.2, Appendix A. Five variants are evaluated in Section 4.1; the description below corresponds to `ret_opt_4_cmp_rsz_buf` the most aggressive variant, which iterates the restructure-then-retime sequence four times. After structural hashing, normalization, and balancing (`st; zero; balance`), a `dretime` pass is applied. The script then iterates the sequence `st; balance; dfraig; dc2 -b -p; drw -z; drf -z; retime -M 5 -o` four times, alternating combinational AIG optimization with mode-5 retiming. A final balancing sequence `(st; balance; dfraig; dc2 -b -p; drw -z)`$\times$`2; balance` is applied as a `compress2`-style phase [12, 15] to further reduce delay and power. Sequential cleanup (`scleanup`) then deletes dead register nodes. Standard cell mapping (`map`) is followed by buffering, sizing, and timing finalization (`topo; stime -c; buffer -c; upsize -c; dnsize -c`). `drw` and `drf` replace `rewrite` and `refactor` due to known stability issues on highly redundant netlists with deep logic [28].

Chapter 3

Methodology

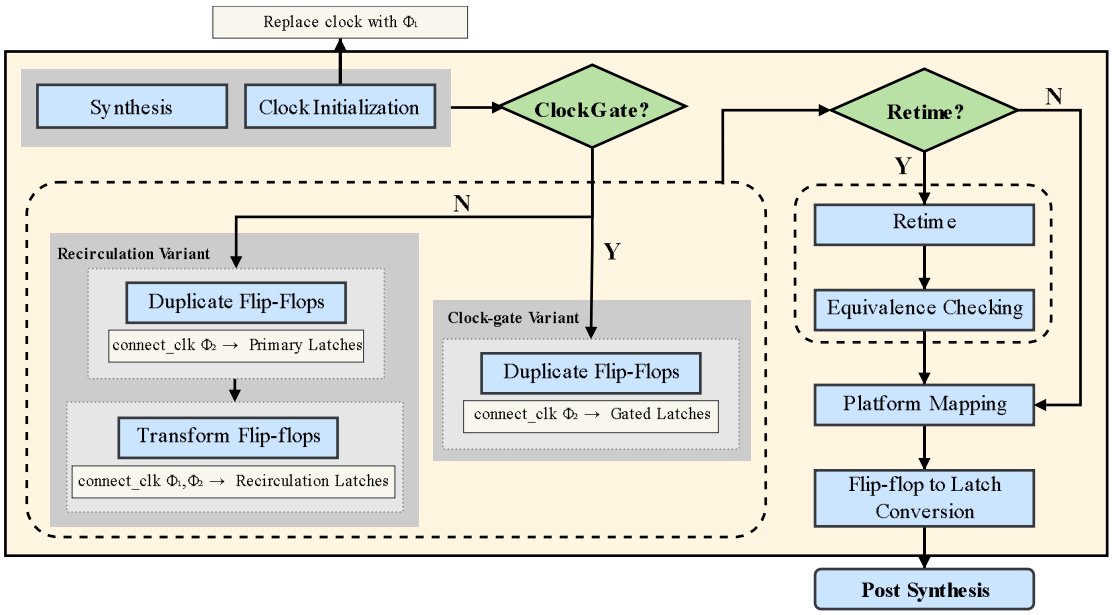


Figure 3.1: Two-phase front-end flow.

3.1 Frontend: Synthesis and Clock Conversion

The front-end uses Yosys `techmap` to duplicate and transform flip-flops, and ABC `retime` to redistribute sequential elements after duplication [26, 4]. The full frontend methodology is presented in Fig. 3.1. The *_DFFE_P_* cell is used, which is a positive edge D-type flip-flop with positive polarity enable, as an example throughout this section to illustrate the transformation to its two-phase clocked counterpart.

This thesis also utilizes the `connect_clk` command developed by Wang that manipulates the RTL netlist by reassigning module ports. The command operates on the post-synthesis netlist by enumerating all instances of a given cell type via Yosys `select` and rewiring each instance's clock pin to the specified top-level clock port using Yosys `connect`; it is called separately for Φ_1 and Φ_2 cell groups, ensuring that $DFF\_base_{\Phi_1}$ instances are driven by *clk_1* and $DFF\_base_{\Phi_2}$ instances by *clk_2* without modifying any combinational logic.

Clock Port Initialization: To initialize a single-phase system into a two-phase system, a second clock must be added to the top-level. Prior to technology mapping, the design undergoes standard Yosys synthesis with two modifications: the existing clock port is renamed to *clk_1* (Φ_1), and a second clock input *clk_2* (Φ_2) is added to the top-level module.

Recirculation Mux Design – Duplicate D Flip-Flops: A Yosys `techmap` is performed across all flip-flops in the design by duplicating each flip-flop to establish the foundation for retiming and for connecting clocks [26]. Duplication means both flops

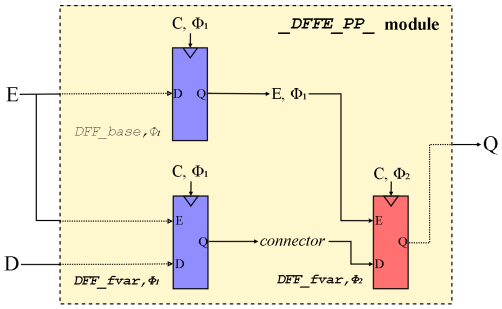


Figure 3.2: Recirculation mux duplication.

in the duplicated pair retain their full variant type (i.e. $DFF\_fvar_{\Phi_1,\Phi_2}$) as illustrated in Fig. 3.2. An additional $DFF\_base_{\Phi_1}$ is inserted for the enable (E) connecting it to the data input of $DFF\_base_{\Phi_1}$, thus making $E \rightarrow E,\Phi_1$ which connects to the second $DFF\_fvar_{\Phi_2}$ control pin(s) to ensure that enable, reset, and set are valid signals fanning out to the correct phase, preventing control signals from propagating between different phases. By construction, E,Φ_2 connects to $DFF\_fvar_{\Phi_1}$, ensuring control signals do not cross phase boundaries.

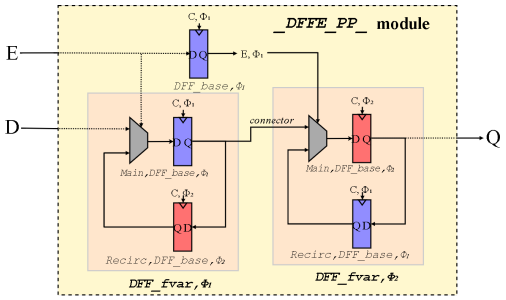


Figure 3.3: Recirculation mux transformation.

Recirculation Mux Design – Transform D Flip-Flops: Since the duplicated flops retain their full-variant type, they require further transformation to

achieve their two-phase latch equivalents, as shown in Fig. 3.3. Both *DFF_fvar*$_{\Phi_1,\Phi_2}$ are transformed into recirculation mux latches. Each *DFF_fvar*$_{\Phi_1,\Phi_2}$ is replaced by a *Main,DFF_base*$_{\Phi_1,\Phi_2}$ and a *Recirc,DFF_base*$_{\Phi_1,\Phi_2}$ pair connected via a 2:1 multiplexer. Input data *D* feeds into the multiplexer, which drives *Main,DFF_base*$_{\Phi_1,\Phi_2}$, while *Recirc,DFF_base*$_{\Phi_1,\Phi_2}$ feeds back to hold the current state. The control pin of the multiplexer determines whether the output updates with new data or recirculates the existing state.

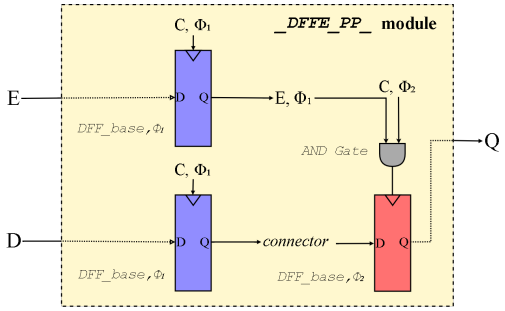


Figure 3.4: Clock-gated duplication/transformation.

Clock-Gated Design – Duplicate and Transform D Flip-Flops: Rather than preserving the full flip-flop variant type, all flip-flop variants are duplicated with an equivalent *DFF_base*$_{\Phi_1,\Phi_2}$. Functional equivalence with the original full-variant flip-flop is achieved by gating the clock input of *DFF_base*$_{\Phi_2}$, as shown in Fig. 3.4. An additional *DFF_base*$_{\Phi_1}$ is required to drive the control pins of *DFF_base*$_{\Phi_2}$ through an *AND* gate, ensuring control signals are correctly phase-aligned.

Retime: As described in Section 2.3, ABC `retime` is applied to the duplicated flip-flop pairs prior to latch transformation. For the recirculation mux design, `retime` is

applied to *DFF_fvar*$_{\Phi_1,\Phi_2}$, redistributing the full-variant flip-flop pairs across combinational logic while maintaining the legal Φ_1/Φ_2 phase separation. For the clock-gated design, `retime` is applied to *DFF_base*$_{\Phi_1,\Phi_2}$, redistributing the base flip-flop pairs across combinational logic.

Equivalence Checking: Yosys equivalence checking [27] is performed on the design before and after retiming to confirm that retiming does not logically alter the circuit.

DFF to Latch: In either variant, a final `techmap` converts all *DFF_base*$_{\Phi_1,\Phi_2}$ into the platform's positive-enable latch.

3.2 Backend: P&R, CTS and Verification

The Φ_1/Φ_2 clocks are connected during clock tree synthesis (CTS) with TritonCTS [19] in OpenROAD [22] with additional options to reduce clock skew, followed by two-phase correctness validation with OpenROAD's Python API [16] to ensure *all* latches are connected to opposite phases via two-coloring check [24].

Clock Tree Synthesis: Independent Φ_1/Φ_2 clock trees are built using `clk_nets` with TritonCTS. The options `balance_levels`, `repair_clock_nets`, and `obstruction_aware` are used together to amend for the different insertion delays for Φ_1/Φ_2 latches. `balance_levels` attempts to keep a similar number of levels in the clock tree across non-register cells (e.g., clock-gate or inverter). `repair_clock_nets` fixes long wires inside CTS prior to latency adjustment with delay buffers, which can lead to a

more balanced clock tree. `obstruction_aware` enables obstruction-aware buffering such that clock buffers are not placed on top of blockages or hard macros.

Two-coloring Static Verification: Two-coloring static verification checks whether a design satisfies two-phase discipline by assigning one color to all Φ_1-derived signals and a second to all Φ_2-derived signals [24]. The coloring propagates through combinational logic, but alternates across latch boundaries, since each latch transfers data from one phase domain to the other. A valid two-phase system must satisfy:

- No latch may have an input and output signal of the same color.
- The latch input signal and clock signal must be of the same color.
- All signals to a combinational logic element must be of the same color.

The first rule ensures that a latch always transfers data between opposite phases, preventing same-phase transparency that could lead to races. The second rule guarantees that the data presented at the latch input is aligned with the clock phase that controls its transparency window. The third rule enforces phase separation in combinational logic so that signals from different clock domains are not mixed within the same logic cone.

This method is applied through OpenDB, a physical design database used within OpenROAD [18] and OpenROAD's API [16], where $G = (V, E)$ is constructed directly from the placed netlist, as shown in Algorithm 1. LATCHGRAPH first populates V by iterating over all instances in the design database and identifying sequential elements. For each sequential element, a DFS is performed backwards along the clock net to the input port to determine whether the element is clocked by Φ_1 or Φ_2. For clock-gated designs,

Algorithm 1 Build Latch Graph

1: **Function** LatchGraph(*design*)
2: **for all** instance u in design **do**
3: **if** u is sequential **then**
4: Determine clock domain of u via backwards DFS on clock net
5: Add u as node to G with color(u) $\leftarrow \Phi_1$ or Φ_2
6: **end if**
7: **end for**
8: **for all** latch node u in G **do**
9: $visited\_nets \leftarrow \emptyset$
10: $F \leftarrow$ DFS through combinational logic from u, marking traversed nets in *visited_nets* and skipping nets already in *visited_nets*
11: **for all** first reachable latch v in F **do**
12: Add directed edge (u, v) to G
13: **if** color(u) = color(v) **then**
14: **Error:** Two-color violation at (u, v)
15: **end if**
16: **end for**
17: **end for**
18: **return** G

the DFS for clock domain accounts for the *AND* gate by tracing through the gate's clock input, ensuring the original clock phase is recovered. A second DFS is then performed forward through combinational logic from each latch stopping at the first reachable sequential element, and each such reachable latch v is added as a directed edge (u, v) to G, producing a structural latch-to-latch connectivity graph suitable for two-coloring verification. Within each DFS traversal from latch u, the *visited_nets* set prevents re-traversal of the same net, ensuring termination in the presence of combinational loops and FSM feedback paths while still allowing G to represent cycles. The two-coloring check is applied after adding the directed edge (u, v) to G; it verifies the graph by checking that for every directed edge (u, v), the clock domains of u and v differ, a single condition that verifies all three of Wolf's two-color constraints [24]. Constraint 1 is directly enforced: every latch boundary transfers data between opposite phase domains. Constraint 2 is implicitly enforced by the stopping condition: because the forward DFS halts at the *first* reachable latch, there is no intermediate latch between u and v, meaning the data signal arriving at v's input originates directly from u's phase domain. Constraint 3 is implicitly checked by LATCHGRAPH construction since edges are built by DFS through combinational logic before stopping at the next latch, preventing cross-phase signal mixing within any logic cone.

Chapter 4

Evaluation

This chapter evaluates the proposed methodology across three experiments. Section 4.1 compares the optimized ABC retime scripts against `abc_speed` and `ret_base` at the logic synthesis stage on the two-phase clock-gated RISCV32I design. Section 4.2 extends this comparison to post-PnR across GCD, AES, RISCV32I, and JPEG, quantifying the area-power and timing-power tradeoffs. Section 4.3 presents the end-to-end comparison between the flip-flop baselines and two-phase variants using the `ret_opt_4_cmp_rsz_buf` optimized retime script.

4.1 Experiment: Logic Synthesis ABC Script Comparison

This section benchmarks the optimized ABC scripts at the logic synthesis step on the two-phase clock-gated RISCV32I design, with results in Table 4.2. RISCV32I is selected as it is the most complete and representative of the four designs. ABC version 1.01 and Yosys version 0.58 (commit 157aabb58) are used. The optimized

Table 4.1: ABC Script Definitions.

ABC Script	**Commands**
abc_speed	&get -n; &st; &dch; &nf; &st; (&syn2; &if -g -K 6; &synch2; &nf; &st)×5; &put; buffer -c; topo; stime -c; upsize -c; dnsize -c
ret_base	strash; zero; dretime; retime -M 5 -o; map
ret_opt_1	st; zero; balance; dretime; st; balance; dfraig; dc2 -b -p; drw -z; drf -z; retime -M 5 -o; map
ret_opt_1_cmp	st; zero; balance; dretime; st; balance; dfraig; dc2 -b -p; drw -z; drf -z; retime -M 5 -o; (st; balance; dfraig; dc2 -b -p; drw -z)×2; balance; scleanup; map
ret_opt_1_cmp_rsz_buf	st; zero; balance; dretime; st; balance; dfraig; dc2 -b -p; drw -z; drf -z; retime -M 5 -o; (st; balance; dfraig; dc2 -b -p; drw -z)×2; balance; scleanup; map; topo; stime -c; buffer -c; upsize -c; dnsize -c
ret_opt_4_cmp	st; zero; balance; dretime; (st; balance; dfraig; dc2 -b -p; drw -z; drf -z; retime -M 5 -o)×4; (st; balance; dfraig; dc2 -b -p; drw -z)×2; balance; scleanup; map
ret_opt_4_cmp_rsz_buf	st; zero; balance; dretime; (st; balance; dfraig; dc2 -b -p; drw -z; drf -z; retime -M 5 -o)×4; (st; balance; dfraig; dc2 -b -p; drw -z)×2; balance; scleanup; map; topo; stime -c; buffer -c; upsize -c; dnsize -c

ABC scripts, `ret_opt_1`, `ret_opt_1_cmp`, `ret_opt_1_cmp_rsz_buf`, `ret_opt_4_cmp`, and `ret_opt_4_cmp_rsz_buf`, are evaluated against `abc_speed` and `ret_base` scripts. The full command sequences for each script are provided in Table 4.1. The Google SkyWater 130nm high-density standard cell library is used for ABC technology mapping. Reported metrics include AIG node and edge counts, max delay, maximum logic level, total cell count, sequential and combinational cell counts, and chip area.

All scripts are invoked at the retiming step of the two-phase front-end flow described in Section 3.1, at which point every flip-flop in the original design has been duplicated into a flip-flop pair. The optimized scripts are organized as a progression of increasingly aggressive transformations on the AIG prior to and after retiming, isolating the contributions of restructuring, post-retime compression, buffer fixing, and iteration depth. `ret_base` represents the simplest case, applying ABC's `retime` command directly without any combinational restructuring, leaving the AIG topology untouched between latch repositioning passes. `ret_opt_1` introduces a single restructure-then-retime pass, inserting `balance`, `dfraig`, `dc2`, and `drw`/`drf` commands before retiming to reshape the AIG and expose new optimization opportunities for retiming. `ret_opt_1_cmp` extends the optimization with a `compress2`-style phase [12] augmented with `dfraig` and `dc2`, with additional `balance`, `dfraig`, `dc2`, and `drw` iterations applied after retiming to further reduce node count and balance logic depth. `ret_opt_1_cmp_rsz_buf` appends a post-mapping resize and buffer-insertion stage (`topo`, `stime -c`, `buffer -c`, `upsize -c`, `dnsize -c`) to fix any residual timing violations introduced by retiming. The four-iteration variants, `ret_opt_4_cmp` and `ret_opt_4_cmp_rsz_buf`, repeat the restructure-

then-retime block four times rather than once, allowing the optimizer to converge on a deeper combinational solution.

Table 4.2: Logic Synthesis Results: Two-phase Clock-gated RISCV32I

Design		Script	Total nd	Total edge	Max delay†	Max lev	Tot. Cells	Seq.	Comb.	Chip Area (μm^2)
RISCV32I	Clock Gated Variant	abc_speed	4126	12298	27.00‡	27	8374	3161	5213	86273.99
		ret_base	13646	28693	2053.12	23	15859	1187	14672	86559.27
		ret_opt_1	12108	22953	3014.93	30	14239	1136	13103	74657.85
		ret_opt_1_cmp	11259	21915	3516.32	39	13390	1136	12254	72523.31
		ret_opt_4_cmp	11681	22218	3486.92	38	13808	1133	12675	74238.70
		ret_opt_1_cmp_rsz_buf	12030	22686	4437.70	42	14161	1136	13025	77103.95
		ret_opt_4_cmp_rsz_buf	12435	22972	4277.32	41	14562	1133	13429	80115.59

†ABC's `delay` field is reported in library delay units for mapped networks.

‡Magnitudes are not directly comparable: `abc_speed` delay is post-mapping; retime variants are pre-resize.

Tot. = Total, Seq. = Sequential Cells, Comb. = Combinational Cells.

4.1.1 ABC Script Comparison: Logic Synthesis Analysis

Logic synthesis results for the two-phase clock-gated RISCV32I variants are presented in Table 4.2.

abc_speed vs Optimized Scripts: The baseline ORFS `abc_speed` script reports the smallest cell count, 8374 cells, 3161 latches, and 5213 combinational cells, with a chip area of 86273.99 μm^2. This is because `abc_speed` does not perform retiming and preserves the position of every duplicated flip-flop-to-latch pair, leaving the surrounding combinational paths in their original form, which were optimized for the original single-phase flip-flop topology rather than for the duplicated two-phase boundary. Compared

to `abc_speed`, the optimized scripts increase total cell count from 8374 to 13390–14562 due to the AIG decomposition required for retiming, but reduce sequential cells from 3161 to 1133–1136 (−64%), outweighing the combinational increase and yielding a net chip area reduction from 86273.99 to 72523.31–80115.59 μm^2 (−7% to −16%).

ret_base vs Optimized Scripts: The original retiming script `ret_base` relocates latches across combinational logic boundaries to take advantage of the additional positional freedom that two-phase flip-flop-to-latch duplication provides. Although `ret_base` applies only retiming (`dretime` and `retime -M 5`) without combinational restructuring, the AIG decomposition required for retiming loses the original combinational structure and inflates the combinational cell count even as the latch count drops. `ret_base` reports 15859 cells (+89%), 1187 latches (−62%), and 14672 combinational cells (+181%), reflecting this restructuring cost when compared to `abc_speed`'s 8374 cells, 3161 latches, and 5213 combinational cells. The optimized scripts show a progressive reduction in total cells and chip area relative to `ret_base`. `ret_opt_1` reduces total cells by 10.2% and chip area by 13.7% with a single restructure-then-retime pass, while `ret_opt_1_cmp` further reduces cells by 15.6% and area by 16.2%, making it the most area-efficient variant. The four-iteration variant `ret_opt_4_cmp` settles slightly higher (−12.9% cells, −14.2% area vs. `ret_base`; +3.1% cells and +2.4% area relative to `ret_opt_1_cmp`), but reduces the sequential count from 1136 to 1133, demonstrating that the addition of the `compress2`-style phase and `scleanup` found a better solution for decreasing sequential cells, even when the final area is not strictly smaller. The `_rsz_buf` variants regress these gains because the post-mapping resize and buffer-insertion stage

adds drive-strength cells to fix hold violations: `ret_opt_1_cmp_rsz_buf` adds +5.8% cells and +6.3% area over `ret_opt_1_cmp`, and `ret_opt_4_cmp_rsz_buf` adds +5.5% cells and +7.9% area over `ret_opt_4_cmp`. The maximum logic level grows from 23 in `ret_base` to 42 in `ret_opt_1_cmp_rsz_buf`, reflecting the depth-latch tradeoff inherent to min-area retiming.

4.2 Experiment: Post-PnR ABC Script Comparison

For the post-PnR experimental setup, the optimized ABC scripts are evaluated against `abc_speed` and `ret_base` on the clock-gated variants using the default ORFS flow with a modified OpenROAD (commit 22a4b4cbaa). Two-phase SDC constraints use 180° waveforms and a 0.49 duty cycle. Default `config.mk` settings are used for each design with the Google SkyWater 130nm high-density library [17] at the nominal TT 25C 1.8V corner. Reported metrics include period, Max TB, Act. TB, Max − Act., hold slack, gated clock power, total power (10% switching activity), sequential, combinational, and total cell counts, and design area.

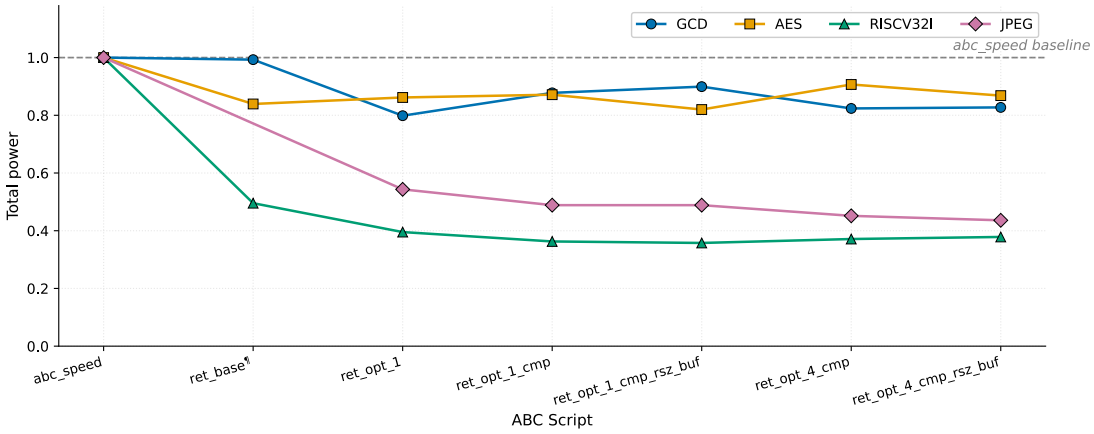
4.2.1 ABC Script Comparison: Post-PnR Analysis

Two-phase clock-gated post-PnR results are presented in Table 4.3, with normalized power and area trajectories in Figs. 4.1a and 4.1b. Setup slack is omitted from the table because it closes to zero across all completed runs, with JPEG `abc_speed` as the sole exception, failing setup with −0.801 ns slack.

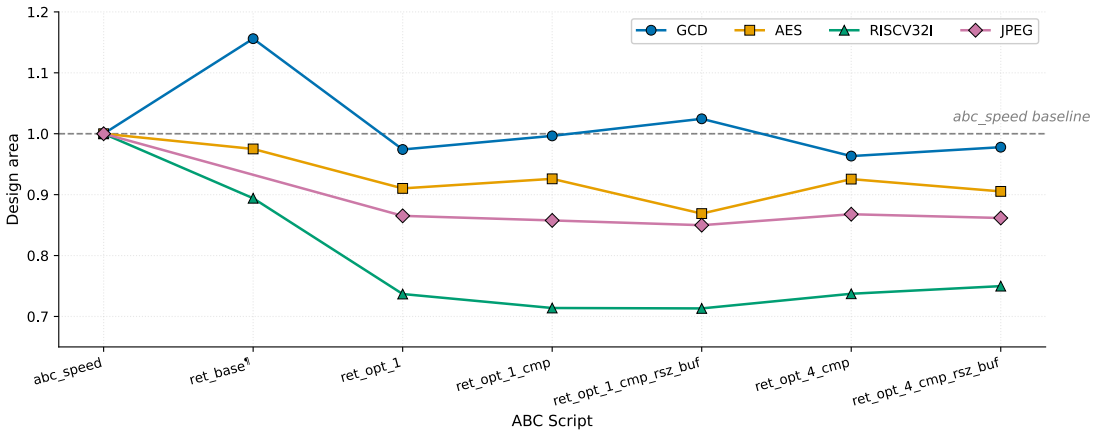
abc_speed vs Optimized Scripts: Every optimized variant reduces total power below the `abc_speed` baseline on every design, with the largest reductions on RISCV32I and JPEG (-64% and -56% respectively) and smaller reductions on GCD and AES (-20% and -18%). On JPEG specifically, `ret_opt_4_cmp_rsz_buf` drops total power from 2190 mW to 955 mW (-56.4%), driven by a -27% reduction in sequential cells once retiming consolidates the latch pairs, and the optimized scripts close `abc_speed`'s setup failure on JPEG.

ret_base vs Optimized Scripts: `ret_opt_1_cmp_rsz_buf` on RISCV32I cuts total power by 27.8% and design area by 20.2% versus `ret_base` while closing hold timing. AES is the outlier, with area improvements ranging from -5.0% to -10.9% versus `ret_base`, total power confined to 2550–2820 mW, and sequential cell count actually increasing from 1179 to 1237. Notably, JPEG `ret_base` failed routing due to congestion from a 100556 combinational cell count, while all optimized ABC scripts route successfully on JPEG with combinational cells reduced to 57636–58738, demonstrating that the AIG restructuring passes produce more route-friendly netlists on large designs where `ret_base` alone does not scale. On GCD specifically, `ret_base` inflates area to 116% of `abc_speed` while the optimized variants pull it back to 96–102%, recovering the area that `ret_base`'s AIG decomposition cost.

Across all designs, timing remains within budget. On RISCV32I, the `_rsz_buf` stage corrects the small hold violations introduced by retiming, with worst hold slack moving from -0.031 ns to 0.005 ns for `ret_opt_1_cmp` and from -0.040 ns to 0.023 ns for `ret_opt_4_cmp` after buffer insertion.



(a) Total power per design trajectory.



(b) Design area per design trajectory.

Figure 4.1: Power and area trajectories across ABC scripts normalized to `abc_speed`.

¶JPEG `ret_base` omitted due to routing failure from congestion.

Table 4.3: ABC Script Comparison: Two-phase Clock-gated Post-PnR Results

All timing values are in ns. All power values are reported with 10% switching activity rate.

Design		Per.	Script	Max TB†	Act. TB†	Max. - Act.§	Hld. Slk	G. Pwr (μW)	Tot. Pwr (mW)	Seq.	Comb.	Tot. Cells	Area (μm^2)
GCD	Clock Gated Variant	3.99	abc_speed	1.73	1.47	0.26	0.102	175.52	2.78	102	219	1165	4249
			ret_base	1.75	0.76	0.99	0.078	154.47	2.76	92	350	1685	4912
			ret_opt_1	1.92	1.25	0.67	0.045	159.17	2.22	75	270	1429	4139
			ret_opt_1_cmp	1.92	1.33	0.59	0.051	156.96	2.44	75	270	1415	4234
			ret_opt_1_cmp_rsz_buf	1.92	1.06	0.86	0.057	156.38	2.50	75	270	1425	4353
			ret_opt_4_cmp	1.92	1.34	0.58	0.050	156.34	2.29	75	268	1386	4093
			ret_opt_4_cmp_rsz_buf	1.76	1.10	0.66	0.049	158.00	2.30	75	268	1382	4155
AES	Clock Gated Variant	4.7	abc_speed	2.22	2.09	0.13	0.069	470.24	3110	1272	10681	71112	130840
			ret_base	2.21	1.88	0.33	0.097	450.00	2610	1179	13251	78882	127572
			ret_opt_1	2.24	1.92	0.32	0.092	452.17	2680	1203	11905	74049	119098
			ret_opt_1_cmp	2.23	1.89	0.34	0.096	454.60	2710	1203	10975	71397	121151
			ret_opt_1_cmp_rsz_buf	2.23	2.02	0.21	0.115	451.45	2550	1203	10975	73402	113685
			ret_opt_4_cmp	2.21	2.00	0.21	0.100	461.30	2820	1237	10855	70889	121092
			ret_opt_4_cmp_rsz_buf	2.06	1.78	0.28	0.141	457.00	2700	1237	10879	73747	118454
RISCV32I	Clock Gated Variant	6.4	abc_speed	2.86	2.55	0.31	0.111	50.22	76.90	3161	4678	27121	104913
			ret_base	2.93	2.48	0.45	-0.004	1.67	38.10	1187	10231	35700	93800
			ret_opt_1	3.11	2.65	0.46	-0.003	1.64	30.40	1136	8471	30096	77305
			ret_opt_1_cmp	3.14	2.61	0.53	-0.031	1.63	27.90	1136	7679	28429	74896
			ret_opt_1_cmp_rsz_buf	2.98	2.33	0.65	0.005	1.63	27.50	1136	7679	29890	74826
			ret_opt_4_cmp	3.14	2.81	0.33	-0.040	1.64	28.55	1133	7581	29203	77349
			ret_opt_4_cmp_rsz_buf	3.08	2.48	0.60	0.023	1.66	29.10	1133	7613	31165	78664
JPEG	Clock Gated Variant	5.5	abc_speed‡	2.36	2.36	0.00	0.069	13100.56	2190	13256	45076	162189	681592
			ret_base*	—	—	—	—	—	—	10298	100556	—	—
			ret_opt_1	2.43	2.02	0.41	0.002	11536.97	1190	9683	57947	206111	589699
			ret_opt_1_cmp	2.60	2.25	0.35	0.084	11645.32	1070	9682	57636	203570	584582
			ret_opt_1_cmp_rsz_buf	2.42	2.19	0.23	0.010	11638.09	1070	9682	57636	205739	579247
			ret_opt_4_cmp	2.59	2.28	0.31	-0.010	11577.28	989	9681	58738	205919	591500
			ret_opt_4_cmp_rsz_buf	2.59	1.86	0.73	0.040	11575.09	955	9681	58738	208324	587303

†Abbreviated as **T**ime **B**orrow. §Max. - Act., computed as Max TB − Act. TB. ‡JPEG `abc_speed` failed setup with −0.801 ns slack. *JPEG `ret_base` not reported due to failures in routing from congestion.

Per. = Period, Hld. Slk = Hold Slack, G. = Gated, Tot. = Total, Seq. = Sequential Cells, Comb. = Combinational Cells, Act. = Actual.

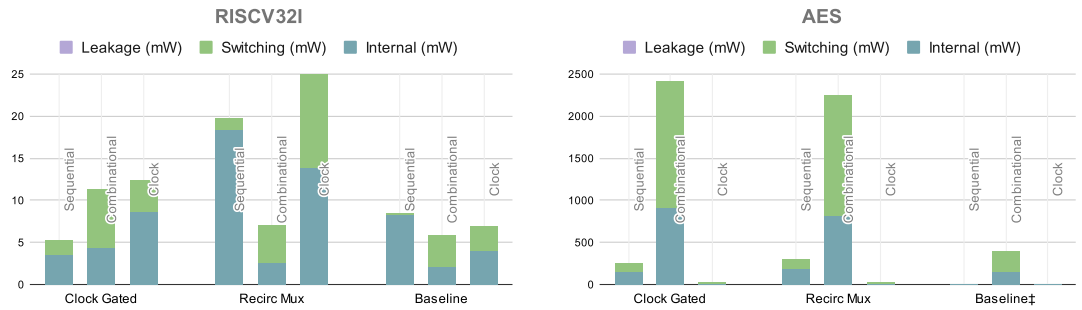
4.3 Experiment: Post-PnR FF vs Two-Phase

In this section, post-PnR results are evaluated similarly to Section 4.2, comparing flip-flop and two-phase variants with the optimized ABC retime script `ret_opt_4_cmp_rsz_buf` from Table 4.1. Reported metrics include total power (with a 10% switching activity rate), clock period, max time borrow, actual time borrow, clock skew, sequential cell count, combinational cell count, total cell count, and design area. The flip-flop baseline designs are synthesized using the default ORFS ABC speed script, which applies standard optimization passes without retiming and instead focuses on optimizing delay meant for flip-flop datapaths.

4.3.1 Post-PnR FF vs Two-Phase: Timing Analysis

Table 4.4 compares the two-phase designs against flip-flop baselines for RISCV32I, GCD, AES, and JPEG at identical target periods after running end-to-end RTL-to-GDS flows. Two-phase clocking enables timing closure through time borrowing at the cost of increased power and latch count. Most notably, the RISCV32I baseline fails timing at 6.4 ns with a setup slack of -0.515 ns, while both two-phase variants close timing at the same period by borrowing up to 2.88 ns. For designs that already meet timing in baseline (GCD, AES), two-phase clocking provides additional timing margin. JPEG's clock-gated variant demonstrates conservative borrowing, using 1.86 ns of the available 2.59 ns, confirming that JPEG has room to increase performance by pushing time borrowing further.

4.3.2 Post-PnR FF vs Two-Phase: Power and Area Analysis



‡Sequential and clock power reported 11.2 mW and 9.07 mW respectively.

Figure 4.2: Power trade-off of two-phase variants over flip-flops of RISCV32I and AES.

Regarding power and area, the cost of timing closure is significant, as shown in Fig. 4.2. The two-phase variants increase total power over the flip-flop baseline through latch duplication and added clock-gating logic, which inflate cell counts and design areas. RISCV32I shows the smallest two-phase penalty (+36.6% for clock-gated, +142.7% for recirculation mux) and the smallest latch count increase (+7.3% for clock-gated), suggesting min-area retiming produced effective results. JPEG follows the same pattern at +96.9% power for clock-gated, while its recirculation mux variant failed routing due to congestion from a 17325-latch count and is omitted.

AES is the outlier. The clock-gated and recirculation mux variants differ in latch count by −22.6% (1237 vs. 1599) but in power by only +5.5%, and all three variants share nearly identical design areas (∼115K–120K μm^2), suggesting that the dominant power contribution in AES comes from its deeply combinational datapath

rather than sequential cell overhead. Dual-clock operation amplifies switching activity through the unrolled cipher rounds, making cell count a poor predictor of power for this design. The elevated clock skew in the RISCV32I clock-gated variant (2.63 ns vs. 0.18 ns for recirculation mux) is attributable to the *AND* gate inserted in the Φ_2 clock path for latch enable gating, which introduces an asymmetric insertion delay relative to the ungated Φ_1 tree.

4.3.3 Clock-Gated vs Recirculation Muxes

Among two-phase variants, clock-gating yields fewer latches and lower power than recirculation mux. For RISCV32I, clock-gating reduces total power by -43.7% (29.1 mW vs. 51.7 mW) and latch count by -73.1% (1133 vs. 4214), with GCD showing a similar trend (-49.2% power, -54.3% latches).

Table 4.4: Experimental Results

All timing values are in ns. All power values are reported with 10% switching activity rate.

Design	Per.	Variant	Max TB†	Act. TB†	Max. − Act.§	Set. Slk	Hld. Slk	Skew	Tot. Pwr (mW)	Seq.	Comb.	Tot. Cells	Area (μm^2)
RISCV32I	6.4	Clock-Gated	3.08	2.48	0.60	0	0.023	2.63	29.1	1133	7613	31165	78664
		Recirc Mux	3.05	2.88	0.17	0	0.286	0.18	51.7	4214	8184	52904	137259
		Baseline	—	—	—	-0.515	0.612	0.07	21.3	1056	3502	18735	70025
GCD	3.99	Clock-Gated	1.76	1.10	0.66	0	0.049	1.16	2.30	75	268	1382	4155
		Recirc Mux	—	—	—	0.101	0.310	0.31	4.53	164	321	2141	5779
		Baseline	—	—	—	0.307	0.333	0.00	1.36	35	187	892	3040
AES	4.7	Clock-Gated	2.06	1.78	0.28	0	0.141	1.60	2700	1237	10879	73747	118454
		Recirc Mux	2.20	1.83	0.37	0	0.225	0.28	2560	1599	10711	77594	120299
		Baseline	—	—	—	0.076	0.045	0.21	420	582	10722	68410	115208
JPEG	5.5	Clock-Gated	2.59	1.86	0.73	0	0.040	2.00	955	9681	58738	208324	587303
		Recirc Mux*	—	—	—	—	—	—	—	17325	89887	—	—
		Baseline	—	—	—	0.037	0.345	0.10	485	4390	27031	118475	452808

†Abbreviated as **T**ime **B**orrow. §Max. − Act., computed as Max. TB − Act. TB. *JPEG recirculation mux variant not reported due to failures in routing from congestion.

Per. = Period, Set. Slk = Setup Slack, Hld. Slk = Hold Slack, Tot. = Total, Seq. = Sequential Cells, Comb. = Combinational Cells.

Chapter 5

Conclusion

This thesis completes the first open-source fully automated two-phase clocking flow in ORFS, extending the front-end flip-flop-to-latch conversion of Wang and Guthaus [23] into an end-to-end RTL-to-GDS flow. The specific contributions introduced here are a clock-gated synthesis variant alongside the existing recirculation-mux variant, an optimized iterative ABC retiming script, dual clock tree synthesis with TritonCTS, and a two-coloring static verification pass implemented through OpenROAD's Python API. The strongest result is RISCV32I clock-gated, which recovers timing closure at 6.4 ns from a failing flip-flop baseline (setup slack of -0.515 ns) at only $+36.6\%$ power overhead being the smallest two-phase power penalty across all four benchmarks. Clock-gating reduces power by -29.2% and latch count by -50.0% on average over recirculation mux, demonstrating that two-phase clocking is viable in an open-source RTL-to-GDS flow despite the power overhead it incurs over flip-flop baselines.

Appendix A

ABC Command Reference

Table A.1: Commands and Flags Used in `ret_base`

Name		Description
Command	`strash/st`	Structural hashing; converts the network to an AIG.
	`zero`	Converts registers to have const-0 initial value.
	`dretime`	Defaults to a min-area retiming pass.
	`retime`	Performs retiming with mode selectable via `-M`.
	`map`	Standard cell technology mapping.
Flag	`-M 3`	Forward and backward min-area retiming.
	`-M 4`	Forward and backward min-delay retiming.
	`-M 5`	Mode 3 followed by mode 4.
	`-o`	Enables using old flop naming conventions.
	`-v`	Verbose output.

Table A.2: Commands and Flags Used in the Optimized Scripts*

	Name	Description
Command	`strash/st`	Structural hashing of the AIG.
	`zero`	Converts registers to have const-0 initial value.
	`balance`	AIG balancing for delay reduction.
	`dretime`	Min-area (or most-forward) retiming. Default min-area.
	`dfraig`	Functionally Reduced AND Inverter Graph (FRAIG).
	`dc2`	Performs combinational optimization.
	`drw`	Performs combinational rewriting.
	`drf`	Performs combinational refactoring.
	`retime`	Retiming with mode selectable via `-M`.
	`scleanup`	Sequential cleanup; removes nodes and latches that do not fanout to primary outputs (POs).
	`topo`	Topologically reorders the netlist.
	`stime`	Static timing analysis under specified `.lib`.
	`buffer`	Inserts buffers using the specified buffer cell.
	`upsize`	Selectively increases gate sizes on the critical path.
	`dnsize`	Selectively decreases gate sizes while maintaining delay.
Flag	`-b`	Enables balancing during `dc2`.
	`-p`	Enables power-aware rewriting during `dc2`.
	`-z`	Zero-cost replacements in `drw`/`drf`; reshapes the AIG even when the immediate node count does not decrease.
	`-M 5`	Mode 3 (forward and backward min-area) followed by mode 4 (forward and backward min-delay) retiming.
	`-o`	Enables using old flop naming conventions.
	`-c`	Toggles use of wire-loads if specified.

*Used in `ret_opt_1`, `ret_opt_1_cmp`, `ret_opt_1_cmp_rsz_buf`, `ret_opt_4_cmp`, and `ret_opt_4_cmp_rsz_buf`.

Table A.3: Optimization Commands and Flags Used in `abc_speed`

Name		Description
Command	`&dch`	Choice computation; builds an AIG with structural choices.
	`&nf`	Priority-cut delay-driven standard cell mapper.
	`&st`	Structural hashing on the GIA.
	`&syn2`	Performs AIG optimization.
	`&if`	Priority-cut mapper; used here for SOP-balancing via `-g`.
	`&synch2`	Synthesis with structural choice regeneration.
	`buffer`	Inserts buffers using the specified buffer cell (e.g. `BUF_X1`).
	`topo`	Topologically reorders the netlist.
	`stime`	Static timing analysis under specified `.lib`.
	`upsize`	Selectively increases gate sizes on the critical path.
	`dnsize`	Selectively decreases gate sizes while maintaining delay.
Flag	`-g`	Enables SOP-balancing in `&if`; decomposes each cut's sum-of-products form into a delay-balanced tree of two-input ANDs.
	`-K 6`	Cut size for `&if` (cuts of up to 6 inputs).
	`-c`	Toggles use of wire-loads (used in `buffer`, `stime`, `upsize`, `dnsize`).

Bibliography

[1] Robert Brummayer and Armin Biere. Local two-level and-inverter graph minimization without blowup. In *Proc. 2nd Doctoral Workshop on Mathematical and Engineering Methods in Computer Science (MEMICS'06)*, Mikulov, Czech Republic, 2006.

[2] Satrajit Chatterjee, Alan Mishchenko, Robert Brayton, Xinning Wang, and Timothy Kam. Reducing structural bias in technology mapping. In *Proc. International Conference on Computer-Aided Design (ICCAD'05)*, pages 519–526, San Jose, California, USA, 2005. IEEE.

[3] A. Hurst et al. The advantages of latch-based design under process variation. In *DAC*. ACM, 2006.

[4] A. Hurst et al. Fast minimum-register retiming via binary maximum-flow. In *IWLS*, 2007.

[5] C. Wolf et al. Yosys – a free verilog synthesis suite. In *Austrochip*, 2013.

[6] D. Harris et al. Timing analysis including clock skew. *TCAD*, 18(11):1608–1618, 1999.

[7] K. Singh et al. Low power latch based design with smart retiming. In *ISQED*, pages 329–334. IEEE, 2018.

[8] R. Brayton et al. ABC: An academic industrial-strength verification tool. In *CAV*, volume 6174 of *LNCS*, pages 24–40. Springer, 2010.

[9] T. Ajayi et al. OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain. *GOMACTECH*, pages 1105–1110, 2019.

[10] E. Friedman. Clock distribution networks in synchronous digital integrated circuits. *Proc. IEEE*, 89(5):665–692, 2001.

[11] M. Horowitz. Lecture 7: Clocking of VLSI systems. Lecture notes, EE271, Stanford University, 2025.

[12] Alan Mishchenko and Robert Brayton. Scalable logic synthesis using a simple circuit structure. In *Proc. International Workshop on Logic Synthesis (IWLS'06)*, pages 15–22, Vail, Colorado, USA, 2006.

[13] Alan Mishchenko, Robert Brayton, Stephen Jang, and Victor Kravets. Delay optimization using SOP balancing. In *Proc. International Conference on Computer-Aided Design (ICCAD'11)*, pages 375–382, San Jose, California, USA, 2011. IEEE.

[14] Alan Mishchenko, Satrajit Chatterjee, and Robert Brayton. DAG-aware AIG rewriting: A fresh look at combinational logic synthesis. In *Proc. 43rd Annual*

Design Automation Conference (DAC'06), pages 532–535, San Francisco, California, USA, 2006. ACM.

[15] Alan Mishchenko, Satrajit Chatterjee, Roland Jiang, and Robert Brayton. FRAIGs: A unifying representation for logic synthesis and verification. Erl technical report, EECS Department, University of California, Berkeley, March 2005.

[16] OpenROAD Project. OpenROAD API. `https://openroad.readthedocs.io/en/latest/main/README2.html`, 2025. Accessed Sept. 2025.

[17] SkyWater Technology. Libraries. `https://skywater-pdk.readthedocs.io/en/main/contents/libraries.html`, 2025. Accessed Aug. 2025.

[18] The OpenROAD Project. OpenDB: Database and tool framework for EDA. `https://github.com/The-OpenROAD-Project/OpenDB/tree/master`, 2019. Accessed: 2026-03-02.

[19] The OpenROAD Project. TritonCTS: Clock tree synthesis. `https://github.com/The-OpenROAD-Project/TritonCTS`, 2025. Accessed Aug. 2025.

[20] The OpenROAD Project. OpenROAD-flow-scripts: ABC area optimization script (`abc_area.script`). `https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts/blob/master/flow/scripts/abc_area.script`, 2026. Accessed: 2026-05-02.

[21] The OpenROAD Project. OpenROAD-flow-scripts: ABC speed optimization script (`abc_speed.script`). `https://github.com/The-OpenROAD-Project/`

OpenROAD-flow-scripts/blob/master/flow/scripts/abc_speed.script, 2026. Accessed: 2026-05-02.

[22] The OpenROAD Project. OpenROAD: RTL-to-GDS flow, 2026.

[23] L. Wang. Revisiting two-phase clocking: Implementing two-phase clocking in openroad flow scripts. Master's thesis, UC Santa Cruz, 2025.

[24] W. Wolf. *Modern VLSI Design: IP-Based Design*. Pearson, Westford, MA, 4 edition, 2009.

[25] YosysHQ. simcells.v. GitHub, 2024.

[26] YosysHQ. techmap — Yosys 0.35 documentation, 2024.

[27] YosysHQ. Equivalence checking. https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd/index_passes_equiv.html, 2025. Accessed Aug. 2025.

[28] YosysHQ Contributors. Comment on issue #4039: ABC assertion Abc_ObjIsNode(pNode) failed — use drw/drf instead of rewrite/refactor. GitHub issue comment, https://github.com/YosysHQ/yosys/issues/4039\#issuecomment-1817937447, 2023. Accessed: 2026-05-02.